

Firebird Bricks

Final Report

Course: CIS 658

Prepared By: Team Firebird Bricks

Team Members:

Benjamin Hannah & Bharghav Bolla

Submitted Date: 04/27/2026

1. Introduction

Firebird Bricks is a modern web-based application developed to showcase LEGO-related content and provide structured interaction between content creators and users. The platform is designed to go beyond traditional content-sharing systems by integrating features such as an ambassador program, consulting services, and direct communication channels within a single, unified interface.

In today's digital landscape, platforms like YouTube enable creators to share content but lack structured mechanisms for deeper engagement, collaboration, and community-building. Firebird Bricks addresses this gap by offering a centralized platform where users can not only explore curated LEGO content but also actively participate through program applications and direct interaction features.

The system is implemented using React with Vite for the frontend, enabling a fast and responsive user interface, and Firebase for backend services such as authentication and potential data management. The application follows a Single Page Application (SPA) architecture, which allows seamless navigation between pages without full page reloads, thereby improving performance and user experience.

The primary goal of this project is to design and develop a scalable, user-friendly platform that enhances content accessibility while enabling structured interaction between creators and users. Through its modular architecture and modern technology stack, Firebird Bricks demonstrates practical implementation of contemporary web development principles, including component-based design, cloud integration, and responsive UI development.

1.1 Problem Statement

In the current digital ecosystem, content creators primarily rely on platforms such as YouTube and social media to share their work. While these platforms are effective for content distribution, they lack structured mechanisms for deeper engagement, collaboration, and community-driven interaction. For LEGO content creators in particular, there is no dedicated platform that combines content showcasing with organized programs such as ambassador initiatives, consulting opportunities, and direct communication channels.

As a result, creators face several challenges:

- Limited ways to formally engage with their audience beyond comments or subscriptions
- Lack of structured programs for collaboration, promotion, or brand representation
- Absence of a centralized platform to manage interactions such as inquiries, applications, and feedback

Similarly, users and viewers who are interested in LEGO content often do not have a streamlined way to:

- explore curated content in an organized manner
- participate in creator-led initiatives
- directly communicate with creators for collaboration or support

These limitations highlight the need for a dedicated platform that integrates content presentation with structured user interaction.

Firebird Bricks addresses this problem by providing a centralized, web-based solution that enables content discovery, structured program participation (such as ambassador and consulting services), and efficient communication through interactive forms. The platform aims to enhance user engagement, improve accessibility, and create a more organized ecosystem for both creators and users.

1.2 Scope & Objectives

Scope

The scope of the Firebird Bricks project includes the design and development of a web-based application that enables users to explore LEGO-related content and interact with the platform through structured features. The system focuses primarily on frontend development using React (with Vite) and integrates Firebase services for authentication and potential backend support.

The project covers the following key areas:

- Development of a responsive and user-friendly interface for displaying LEGO content
- Implementation of multiple pages, including Home, About, Ambassador, Consulting, and Contact
- Integration of navigation using React Router to support a Single Page Application (SPA) experience
- Inclusion of interactive forms for user engagement, such as ambassador applications and contact submissions
- Embedding of external media content (e.g., YouTube videos) for enhanced user experience

Objectives

The primary objectives of the Firebird Bricks system are:

1. **To provide a centralized platform** for showcasing LEGO content in a visually appealing and organized manner.
2. **To enhance user engagement** by enabling structured interaction through features such as ambassador programs, consulting services, and contact forms.
3. **To ensure smooth navigation and performance** by implementing a Single Page Application (SPA) architecture using React and Vite.
4. **To maintain scalability and flexibility** by integrating Firebase services, allowing the system to support future enhancements such as database storage and authentication expansion.

5. **To deliver a responsive and user-friendly interface** that works seamlessly across different devices and screen sizes.
6. **To demonstrate modern web development practices**, including component-based design, modular architecture, and cloud integration.

Stakeholders are individuals or groups who interact with or are impacted by the Firebird Bricks platform. Identifying stakeholders helps ensure that the system is designed to meet user needs and project requirements effectively.

1.3 Stakeholders

1. Primary Users (Content Viewers)

These are users who visit the platform to explore LEGO-related content. They interact with the system by:

- browsing different pages such as Home and About
- watching embedded videos
- exploring featured or popular content

They benefit from a clean interface and easy navigation that enhances content discovery.

2. Content Creators (Firebird Bricks Owner/Team)

Content creators are responsible for producing and showcasing LEGO builds and related media. They benefit from:

- a centralized platform to present their work
- structured engagement features such as ambassador programs and consulting services
- a professional interface to interact with their audience

3. Applicants (Ambassador Program Users)

These users interact with the platform by applying to become ambassadors. They:

- provide relevant details for evaluation
- engage with the platform beyond passive viewing

They benefit from a structured and easy-to-use application process.

2. System Architecture

2.1 Architecture Overview

The Firebird Bricks system follows a **client-server architecture** with a frontend-focused design. The application is implemented as a **Single Page Application (SPA)** using React (with Vite), while Firebase is used to provide backend services such as authentication and potential data storage.

The architecture is designed to ensure:

- fast performance
- smooth navigation
- scalability through cloud-based services

The system integrates three main layers:

- Frontend Layer (React UI)
- Backend Layer (Firebase Services)
- External Services (YouTube integration)

2.2 High-Level Architecture Components

Frontend Layer (React + Vite)

The frontend is responsible for rendering the user interface and managing all user interactions. It is built using React functional components and uses React Router for navigation between pages.

Key responsibilities include:

- displaying content across multiple pages (Home, About, Ambassador, Consulting, Contact)
- handling user inputs through forms
- managing application state
- enabling dynamic UI updates without page reloads

The use of Vite ensures fast build times and efficient development performance.

Backend Layer (Firebase)

Firebase provides backend functionality in a serverless environment. It removes the need for managing custom servers and enables scalable cloud-based operations.

Key functionalities include:

- **Authentication:** Secure user identity management

- **Database (optional):** Storage of form data such as contact messages and applications
- **Hosting (optional):** Deployment support

Firebase allows the application to scale without additional backend infrastructure.

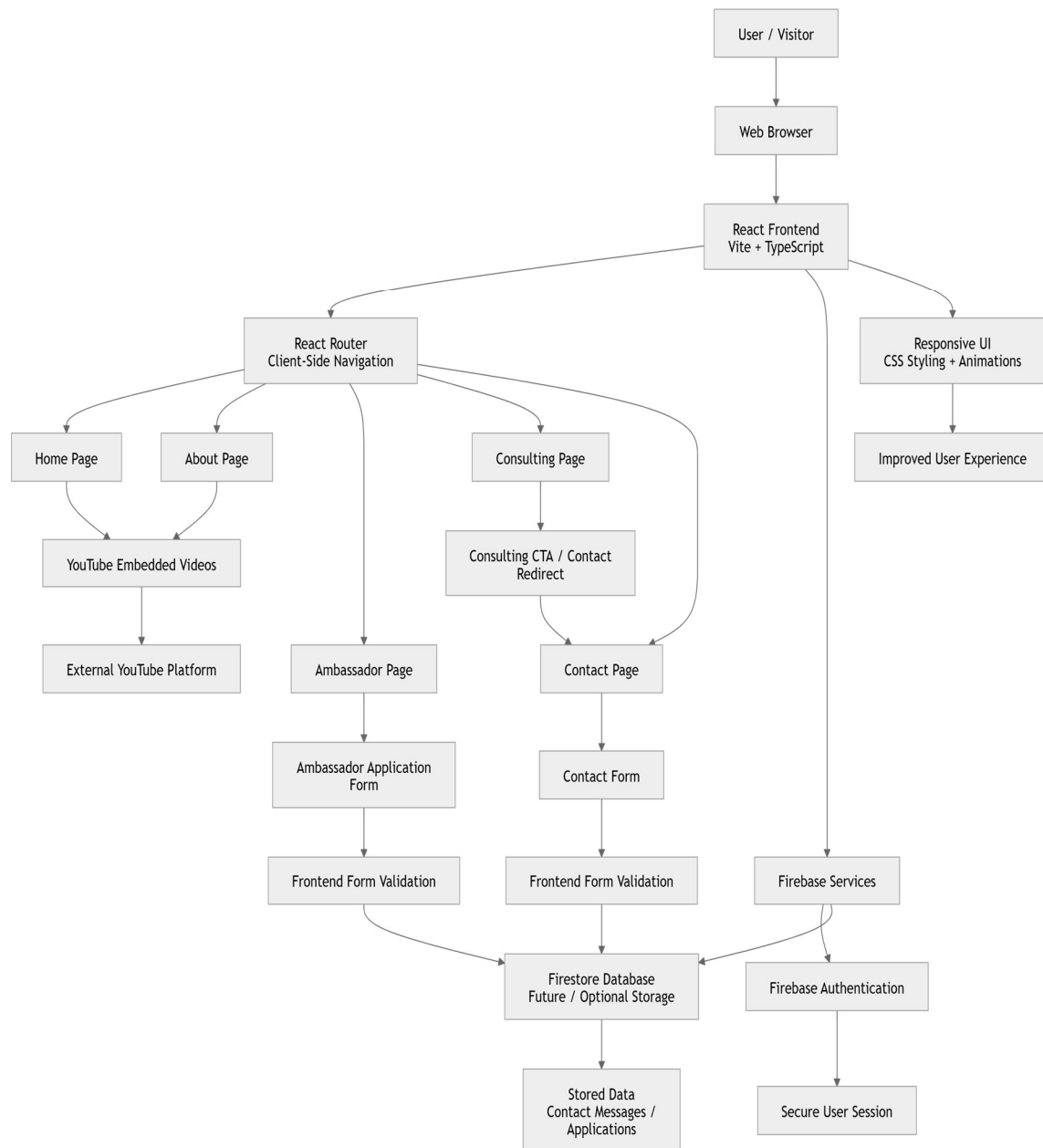
External Integration Layer

The system integrates external services to enhance functionality:

- **YouTube (iframe embedding):**
Used to display LEGO-related video content directly within the application

This allows content to be streamed efficiently without storing large media files in the system.

2.3 System Architecture Diagram



The Firebird Bricks system follows a client-server architecture. Users access the application through a web browser, where the React frontend built with Vite renders the user interface. React Router manages navigation between pages such as Home, About, Ambassador, Consulting, and Contact. Firebase provides backend services such as authentication and optional Firestore data storage. External YouTube embeds are used to display video content without storing media files inside the application.

2.4 Data Flow

User Interaction Flow (Forms)

1. User navigates to a page (Ambassador that navigates to other pages)
2. User enters input
3. React captures input using state management
4. Input validation is performed on the frontend
5. A success message is displayed
6. (Future scope) Data is stored in Firebase database

Navigation Flow

1. User selects a navigation option (e.g., About, Consulting)
2. React Router updates the URL
3. The corresponding page component is rendered
4. The page loads instantly without full reload

Content Flow (Videos)

1. User accesses a page containing video content
2. Embedded YouTube iframe loads the video
3. Video is streamed directly from YouTube

2.5 Architectural Characteristics

The system demonstrates the following architectural qualities:

- **Modularity:** Components are separated into pages and reusable UI sections
- **Scalability:** Firebase allows backend services to scale automatically
- **Performance:** SPA architecture avoids unnecessary reloads
- **Maintainability:** Component-based design simplifies updates

3. Requirements

3.1 Functional Requirements

Functional requirements describe what the system **must do**.

The Firebird Bricks system provides the following core functionalities:

1. Page Navigation

- Users must be able to navigate between different pages:
 - Home
 - About
 - Ambassador
 - Consulting
 - Contact
- Navigation should be handled without full page reloads (SPA behavior)

2. View Content

- Users can view LEGO-related content on the platform
- Embedded YouTube videos should be accessible directly within the site
- Content should be displayed in an organized and visually appealing layout

3. Ambassador

- Users must be able to:
 - access the Ambassador page
 - navigate to consulting page
 - submit their details
- The system should:
 - validate required fields
 - provide feedback after submission

4. Consulting Interaction

- Users should be able to:
 - view consulting services information
 - follow a defined process (steps/cards UI)
 - navigate to contact page for further interaction

5. Contact Form

- Users must be able to:
 - enter name, email, subject, and message
 - submit the form

- The system should:
 - validate inputs
 - display a success message after submission

6. External Media Integration

- The system must support:
 - embedding YouTube videos using iframe
 - seamless playback within the application

7. Firebase Integration

- The system must:
 - integrate Firebase services
 - support authentication (if enabled)
 - allow future data storage in Firestore

3.2 Non-Functional Requirements

Non-functional requirements define **how well the system performs**.

1. Usability

- The interface should be:
 - simple
 - clean
 - easy to navigate
- Users should interact without prior training

2. Performance

- Pages should load quickly using Vite
- Navigation should be instant (SPA)
- UI rendering should be smooth

3. Security

- Firebase Authentication ensures secure user handling
- API keys must be stored in .env files
- User data must not be exposed

4. Scalability

- Firebase allows automatic backend scaling
- React component structure supports easy feature expansion
- New pages or features can be added without redesign

5. Reliability

- The system should:
 - handle navigation without crashes
 - provide consistent UI behavior
 - handle form submissions correctly

6. Maintainability

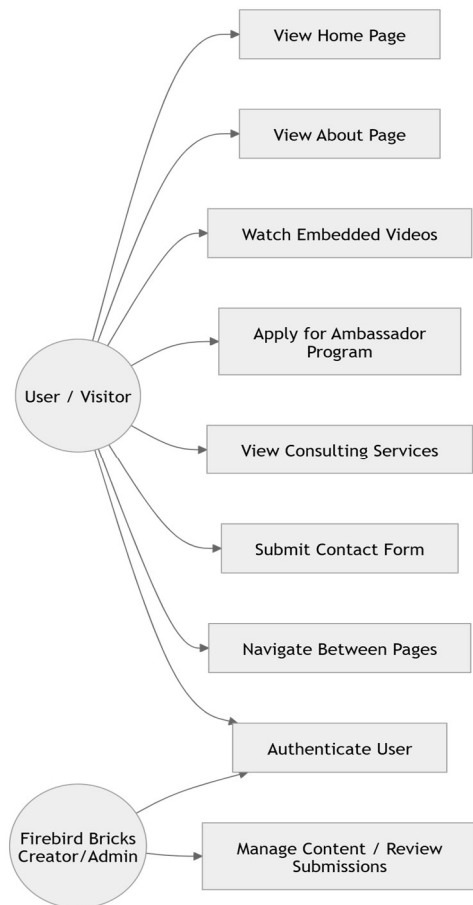
- Code should be:
 - modular
 - readable
 - easy to update

7. Compatibility

- The system should:
 - work on modern browsers (Chrome, Edge, Firefox)
 - support different screen sizes (responsive design)

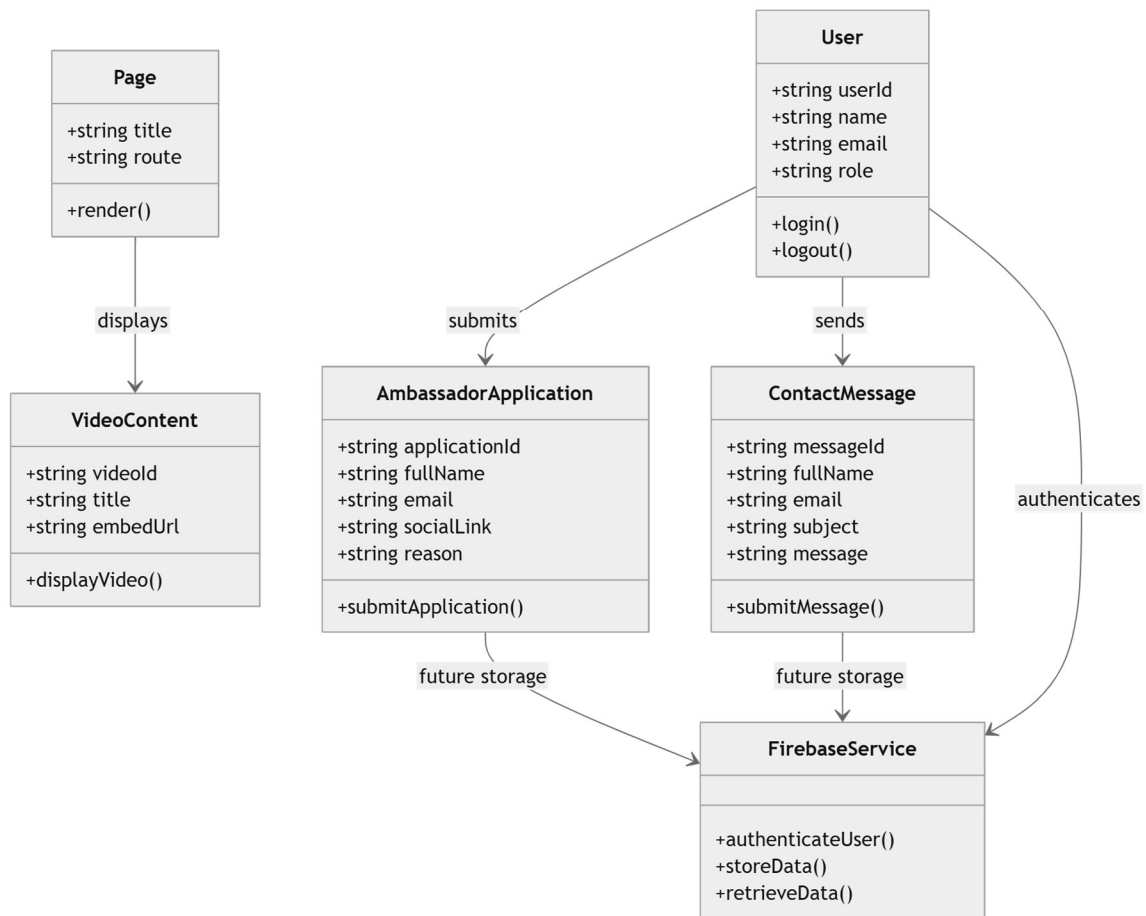
3.3 UML Diagrams

3.3.1 Use Case Diagram



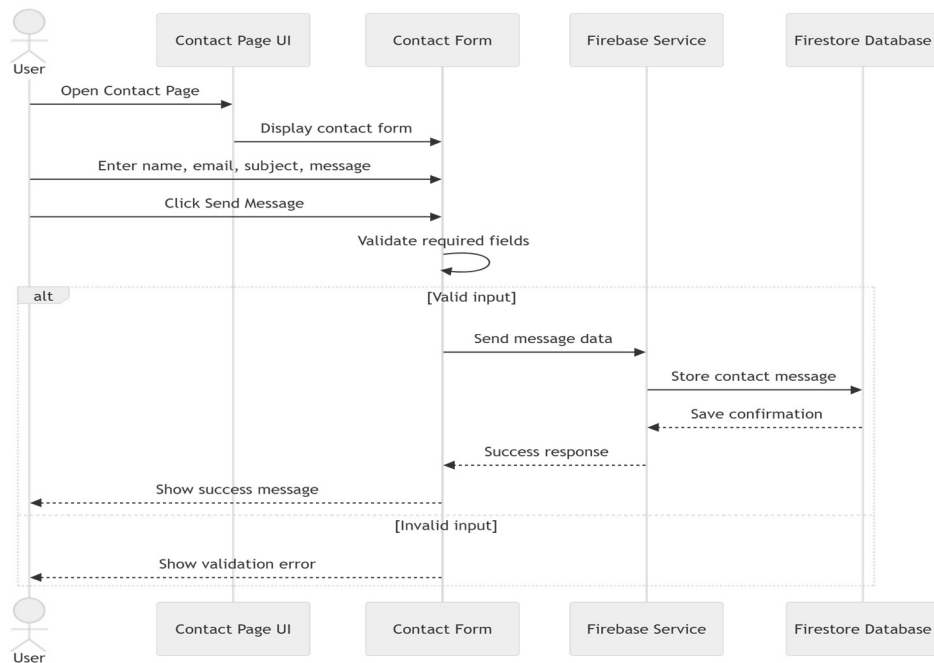
The use case diagram shows how users interact with the Firebird Bricks platform. Users can view content, watch videos, apply for the ambassador program, access consulting services, and submit contact messages. The creator or admin can authenticate and manage platform-related content or submissions.

3.3.2 Class Diagram



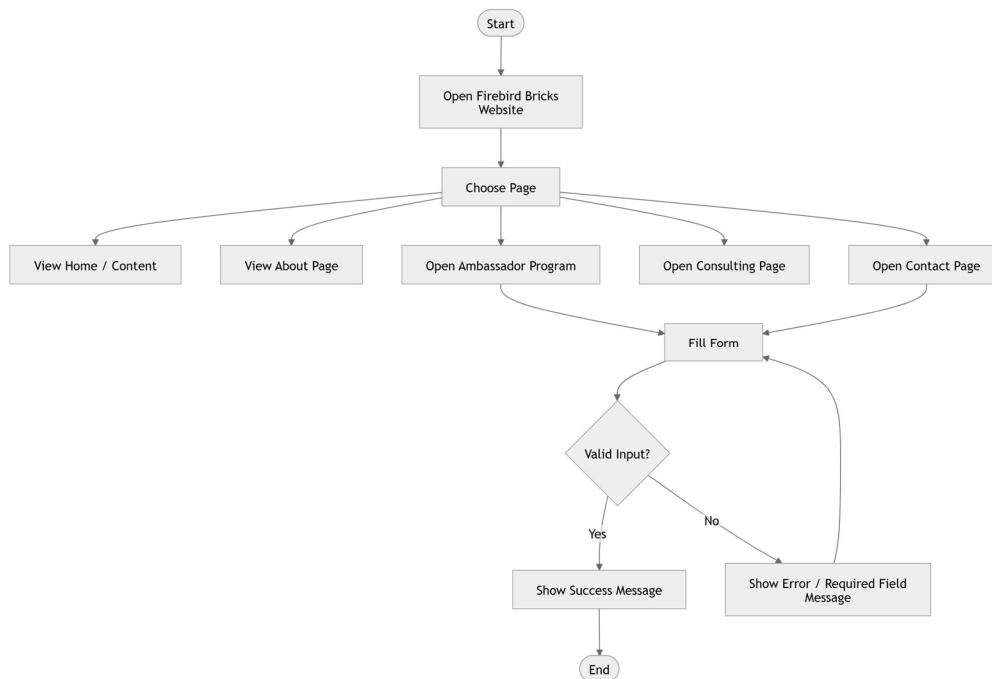
The class diagram represents the conceptual structure of the Firebird Bricks system. The User interacts with pages, submits ambassador applications, and sends contact messages. FirebaseService represents backend services such as authentication and future data storage.

3.3.3 Sequence Diagram — Contact Form Submission



This sequence diagram shows the contact form workflow. The user fills out the form, the frontend validates the input, and the data can be sent to Firebase for storage. After successful submission, the system displays a success message.

3.3.5 Activity Diagram



The activity diagram shows the general user workflow. A user enters the website, navigates to different pages, and may submit forms through the Ambassador or Contact

pages. The system validates inputs and provides either success feedback or error messages.

3.4 Traceability of Requirements to Design

To ensure that the Firebird Bricks system satisfies its intended requirements, each major functional and non-functional requirement is mapped to specific design elements, UI pages, and system components. This traceability helps confirm that every requirement is supported by the system design and implementation.

R1 – Users can navigate between pages

- **Requirement:** Users should be able to move between Home, About, Ambassador, Consulting, and Contact pages.
- **Design Support:** React Router manages client-side navigation.
- **UI Components:** Home page, About page, Ambassador page, Consulting page, Contact page.
- **Related Diagram:** Use Case Diagram – “Navigate Between Pages.”
- **Implementation:** Routes are defined for /, /about, /ambassador, /consulting, and /contact.

R2 – Users can view LEGO-related content

- **Requirement:** Users should be able to view LEGO content and videos.
- **Design Support:** Embedded YouTube videos are displayed using iframe components.
- **UI Components:** About page and video sections.
- **Related Diagram:** Use Case Diagram – “Watch Embedded Videos.”
- **Implementation:** YouTube embed links are used to display selected Firebird Bricks videos.

R5 – Users can access consulting services

- **Requirement:** Users should be able to view consulting information and continue to the contact page.
- **Design Support:** Consulting page explains services, testimonials, and process steps.
- **UI Components:** Consulting page and Contact button.
- **Related Diagram:** Use Case Diagram – “View Consulting Services.”

- **Implementation:** A call-to-action button redirects users from Consulting to Contact.

R6– Users can submit a contact message

- **Requirement:** Users should be able to send messages through the Contact page.
- **Design Support:** Contact page includes a contact form with validation and success feedback.
- **UI Components:** Contact form section.
- **Related Diagram:** Sequence Diagram – “Contact Form Submission.”
- **Implementation:** The form requires name, email, subject, and message before submission.

R7 – System should support Firebase integration

- **Requirement:** The system should integrate Firebase for backend support.
- **Design Support:** Firebase services are used for authentication and future data storage.
- **UI Components:** Authentication-related logic and form submission workflows.
- **Related Diagram:** Class Diagram – FirebaseService.
- **Implementation:** Firebase configuration is included in the project and connected through environment variables.

R8 – System should be user-friendly and responsive

- **Requirement:** The system should be easy to use and work on different screen sizes.
- **Design Support:** CSS Grid, Flexbox, responsive media queries, and consistent layouts.
- **UI Components:** All pages.
- **Related Diagram:** Activity Diagram – User workflow.
- **Implementation:** Responsive sections stack properly on smaller screens.

R9 – System should be maintainable and scalable

- **Requirement:** The codebase should support future improvements.
- **Design Support:** React page-based structure and Firebase backend services.
- **UI Components:** Modular page files and reusable styling patterns.

- **Related Diagram:** Class Diagram and System Architecture Diagram.
- **Implementation:** New pages, forms, and services can be added without redesigning the full system.

This mapping ensures that the Firebird Bricks design is aligned with the system requirements. It also provides a checklist for future development, allowing the team to verify that new features are connected to user needs, interface components, and architectural decisions.

4. Software Design

4.1 Core Concepts of the System

The system is built around the following main conceptual elements:

1. User

The User represents any individual interacting with the platform. This includes:

- general visitors (viewing content)
- applicants (submitting ambassador forms)
- clients (using consulting services)

Key attributes:

- name
- email
- role (optional: visitor, applicant, admin)

Responsibilities:

- navigate the platform
- submit forms
- interact with content

2. Page

The Page represents different sections of the application.

Examples:

- Home
- About

- Ambassador
- Consulting
- Contact

Each page is responsible for:

- displaying specific content
- managing layout
- handling user interaction

3. VideoContent

Represents LEGO-related video content displayed in the system.

Attributes:

- video ID
- title
- embed URL

Responsibilities:

- display embedded YouTube videos
- provide interactive viewing

4. FirebaseService

Represents backend interaction through Firebase.

Responsibilities:

- authentication handling
- storing/retrieving data (future scope)
- managing backend communication

4.2 Architectural Design Approach

The system follows a layered and modular design, where each layer has a specific responsibility:

Frontend Layer

- built using React components

- handles UI rendering
- manages routing and state

Service Layer

- represented by Firebase services
- handles authentication and backend operations

External Integration Layer

- integrates YouTube for video streaming

This separation ensures:

- clean code structure
- easy maintenance
- scalability

4.3 Design Principles Used

The system is designed using key software engineering principles:

1. Separation of Concerns

Each part of the system handles a specific responsibility:

- UI → React
- backend → Firebase
- content → external APIs

2. Modularity

- pages are independent components
- features can be added without affecting others

3. Reusability

- UI elements and styles can be reused
- consistent layout across pages

4.4 Design Patterns Used

Although not implemented explicitly as formal classes, the system reflects common design patterns:

1. Component-Based Pattern

- React components represent reusable UI elements

- improves maintainability

2. Service Pattern

- Firebase acts as a backend service provider
- separates frontend and backend logic

3. Observer Pattern

- React state updates trigger UI re-rendering
- ensures dynamic interaction

4. Routing Pattern

- React Router handles navigation
- enables SPA behavior

5. Implementation & Testing

5.1 Technology Stack

The Firebird Bricks system is developed using a modern frontend-focused technology stack combined with cloud-based backend services.

Frontend Technologies

- **React (with TypeScript):**
Used to build a dynamic and component-based user interface
- **Vite:**
Provides fast build times and efficient development workflow
- **React Router:**
Enables navigation between pages without full page reloads
- **CSS (Flexbox & Grid):**
Used for responsive and structured layout design

Backend Technologies

- **Firebase Authentication:**
Handles user authentication and session management
- **Firebase Firestore (optional):**
Can be used to store form submissions such as contact messages and ambassador applications

External Services

- **YouTube (iframe embedding):**
Used to display LEGO-related video content within the platform

5.2 Implementation Highlights

1. Multi-Page Navigation

The application uses React Router to create a Single Page Application (SPA).

Implemented Pages:

- Home
- About
- Ambassador
- Consulting
- Contact

Navigation is handled without full page reloads, ensuring a smooth user experience.

2. User Interface Design

- Dark-themed modern UI
- Gradient buttons and hover effects
- Responsive layout using CSS Grid and Flexbox
- Consistent styling across all pages

4. Firebase Integration

Firebase is integrated into the system for:

- authentication support
- potential data storage (future scope)

Environment variables are used to securely manage API keys.

5. External Media Integration

YouTube videos are embedded using iframe elements, allowing:

- seamless video playback
- reduced storage requirements

- improved performance

5.3 Testing Strategy

Testing ensures that the system behaves correctly and provides a smooth user experience.

5.3.1 Functional Testing

- Verified navigation between pages
- Tested form submissions
- Ensured proper validation of inputs

5.3.2 User Interface Testing

- Checked layout consistency across pages
- Tested responsiveness on different screen sizes
- Verified styling and visual elements

4.3.3 Integration Testing

- Tested interaction between React frontend and Firebase services
- Verified that forms and authentication components function correctly

5.3.4 Manual Testing

Manual testing was performed throughout development to:

- identify UI issues
- fix navigation errors
- improve user experience

5.4 Challenges Faced

- Initial routing issues causing blank pages
- Firebase configuration errors during setup
- UI consistency across multiple pages
- managing collaboration changes via Git

These challenges were resolved through iterative debugging and testing.

5.5 Future Improvements in Testing

- Implement automated testing (Jest / React Testing Library)
- Add end-to-end testing (Cypress or Playwright)
- Improve error handling and validation

6. Final Reflections and Conclusion

6.1 What Went Well

The development of the Firebird Bricks system demonstrated several successful outcomes:

- **Smooth Navigation Experience:**
Implementing React Router enabled seamless page transitions without full reloads, significantly improving user experience.
- **Effective UI Design:**
The platform achieved a modern and visually consistent interface using a dark theme, gradient elements, and responsive layouts. This contributed to better usability and overall presentation.
- **Successful Integration of External Content:**
Embedding YouTube videos allowed efficient content delivery without increasing storage complexity.
- **Collaborative Development Workflow:**
Using GitHub for version control supported teamwork, enabling code sharing, updates, and issue resolution throughout the project.

6.2 Challenges Encountered

During development, several challenges were identified and addressed:

- **Routing Issues:**
Initial problems with React Router caused blank pages or incorrect navigation, which required debugging and proper route configuration.
- **Firebase Configuration:**
Setting up Firebase services and managing environment variables caused errors initially, especially when synchronizing configurations across team members.
- **UI Consistency:**
Maintaining consistent design across multiple pages required careful adjustment of styles and layout components.
- **Team Integration:**
Changes made by different team members sometimes led to dependency or configuration issues, requiring additional setup and troubleshooting.

6.3 Lessons Learned

The project provided valuable insights into real-world software development:

- Importance of proper project structure and modular design
- Need for consistent UI/UX design across all pages
- Benefits of using cloud-based backend services like Firebase
- Importance of communication and synchronization in team development
- Value of iterative testing and debugging

6.4 Future Enhancements

The Firebird Bricks system has strong potential for further improvement. Future enhancements may include:

- **Full Backend Integration:**
Submitting contact information in Firebase Firestore
- **User Authentication System:**
Expanding login functionality with role-based access
- **Admin Dashboard:**
Allowing administrators to manage content, applications, and messages
- **Advanced Features:**
 - analytics dashboard
 - recommendation system
 - improved UI animations
- **Automated Testing:**
Implementing testing frameworks for reliability

7. Conclusion

The Firebird Bricks project successfully demonstrates the application of modern web development technologies and design principles. By combining React, Vite, and Firebase, the system provides a scalable, efficient, and user-friendly platform for LEGO content presentation and interaction.

The project not only meets its functional requirements but also addresses key non-functional aspects such as performance, usability, and scalability. Through iterative development and problem-solving, the system evolved into a well-structured application that reflects real-world development practices.

Overall, Firebird Bricks serves as a strong foundation for future enhancements and showcases the practical implementation of software architecture, system design, and frontend-backend integration in a collaborative development environment.